

Sockets - Network Programming

? What Are Sockets?

A socket is an endpoint for network communication between processes or machines.

- **TCP (SOCK_STREAM)**: reliable, ordered byte-stream
- **UDP (SOCK_DGRAM)**: unreliable, message-based
- Each socket uses a file descriptor (fd)
- Data must be in network byte order (htons/htonl)
- Non-blocking sockets + select/poll allow handling multiple clients
- **In Webserv**: sockets are used to accept clients, read requests, send responses

Header/Import

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
#include <fcntl.h>
#include <cstring>
#include <iostream>
#include <vector>
#include <errno.h>
```

Declaration/Syntax

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in address;
address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
```

```
address.sin_port = htons(8080);
```

🔧 Common Methods/Usage

- `socket()` - create socket
- `bind()` - attach socket to IP + port
- `listen()` - prepare to accept connections
- `accept()` - accept a client, returns fd
- `recv()/read()` - receive data
- `send()/write()` - send data
- `close()` - close socket
- `fcntl()` - set non-blocking
- `select()/poll()` - multiplex multiple fds

🔧 Parameters / Arguments

socket(int domain, int type, int protocol)

- `domain`: AF_INET (IPv4), AF_INET6 (IPv6), AF_UNIX (local)
- `type`: SOCK_STREAM (TCP), SOCK_DGRAM (UDP)
- `protocol`: usually 0

bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen)

- `sockfd`: fd from socket()
- `addr`: pointer to address struct (sockaddr_in)
- `addrlen`: size of struct (sizeof(sockaddr_in))

listen(int sockfd, int backlog)

- `sockfd`: socket fd
- `backlog`: max pending connections

accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen)

- `sockfd`: listening socket
- `addr`: pointer to client address (can be NULL)
- `addrlen`: pointer to size of `addr` (can be NULL)
- returns: new fd for client connection

`recv(int sockfd, void *buf, size_t len, int flags)`

- `sockfd`: client fd
- `buf`: memory to store data
- `len`: max bytes to read
- `flags`: usually 0
- returns: number of bytes read, 0 = client closed

`send(int sockfd, const void *buf, size_t len, int flags)`

- `sockfd`: client fd
- `buf`: memory to send
- `len`: number of bytes
- `flags`: usually 0
- returns: number of bytes sent



Basic Example

```
int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(8080);

    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 10);

    int client_fd = accept(server_fd, NULL, NULL);
    char buffer[1024] = {0};
    recv(client_fd, buffer, sizeof(buffer)-1, 0);
    const char* response = "HTTP/1.1 200 OK\r\n\r\nHello World";
```

```

    send(client_fd, response, strlen(response), 0);

    close(client_fd);
    close(server_fd);
    return 0;
}

```



Advanced Example (Multiple Clients with Select)

```

std::vector<int> clients;
fd_set read_fds;
int max_fd = server_fd;

while (true) {
    FD_ZERO(&read_fds);
    FD_SET(server_fd, &read_fds);

    for (size_t i = 0; i < clients.size(); i++) {
        FD_SET(clients[i], &read_fds);
        if (clients[i] > max_fd)
            max_fd = clients[i];
    }

    int activity = select(max_fd + 1, &read_fds, NULL, NULL, NULL);

    if (FD_ISSET(server_fd, &read_fds)) {
        int client_fd = accept(server_fd, NULL, NULL);
        clients.push_back(client_fd);
    }

    for (size_t i = 0; i < clients.size(); i++) {
        if (FD_ISSET(clients[i], &read_fds)) {
            char buffer[1024];
            int bytes = recv(clients[i], buffer, sizeof(buffer), 0);
            if (bytes <= 0) {
                close(clients[i]);
                clients.erase(clients.begin() + i);
            }
        }
    }
}

```



42 Specific Notes

- **Norm:** max 25 lines per function, no magic numbers
- Always check return codes
- Store clients in `vector<int>` or `map<int, Client>`

- Non-blocking + select recommended
- Parse HTTP requests correctly, handle partial reads/writes
- Handle SIGPIPE or use MSG_NOSIGNAL

Gotchas

- Forgetting `htons/htonl` → wrong port/address
- Partial reads/writes: always loop until complete
- Iterator invalidation when erasing clients from vector
- `recv()` returns 0 = client closed
- Non-blocking + select/poll is safer

CPP98 Constraints

- No auto, no range-based for loops
- No initializer lists {1,2,3}
- No smart pointers: manual memory management

Related

HTTP, CGI, select, poll, fcntl, sockaddr_in, inet_ntoa, vector